Experiment no: 1

1. WAP to declare a class student having data members as roll, name, accept & data for One object.

→
```
#include <iostream>
using namespace std;
class student{
Private:
string name;
int roll;
Public:
Void accept()
{
cout << "enter student name \n";
cin >> name;
cout << "enter roll no. \n";
cin >> roll;
}
void display()
{
cout << "student name:" << name;
cout << "\n roll no.:" << roll;
}
};
int main()
{
Student S1;
S1. accept();
S1. display();
}
```

Output:

Enter student name:
Sai
Enter roll no:
42
Student name: Sai
Roll no: 42

2. WAP to declare a class having data members as bid, name, price. Accept data for 2 Books & display name of book having greater price.

→
```
#include <iostream>
using namespace std;
class book {
private:
string name;
float id;
public:
float price;
void accept() {
cout << "enter book id \n";
cin >> "id";
cout << "enter book name \n";
cin >> "name";
cout << "enter price \n";
cin >> "price";
}
```

```
void disp (){
cout << "Book id :" << id;
cout<< "Book name :" << name;
cout<< "Book price :" << price;
}
};
int main{
book b1, b2;
b1. accept ();
b2. accept ();
b1. disp ();
b2. disp ();
if ( b1. price > b2.price)
cout<< "\n book 1 price is more";
else
cout<< "\n book 2 price is more";
}
```

output:

```
Enter book id: 123
Enter book name: abc
Enter book price: 986
Enter book id :456
Enter book name : xyz
enter book price : 1065

Book name: abc
Book id :123
Book price : 986
Book name : xyz
Book id :456
Book price :1065
Book 2 price is more
```

Q3. WAP to declare class time. Accept time in HH:MM:SS & display total time in secs.

→

```
# include <iostream>
using namespace std;
class time
{
Public:
float hr, min, sec;
float sum;

void accept (){
cout<< "enter hours";
cin>> hr";
cout << "enter minutes: ";
cin >>" mins ";
cout<< " enter seconds";
cin >> " sec ";
}
void display ()
{
cout << " Hours: "<< hr<<\n;
cout << " minutes:" << min<<\n;
cout << "seconds: "<<sec<<\n;
}
void to second ()
{
```

```
Sum=((hr*3600)+(min*60)+sec);
cout << " Total seconds : " << sum << "\n";
}
};
int main(){
time t1;
t1.accept();
t1.disp();
t1.to seconds;
return 0;
}
```

Output :

```
Enter hour : 1
Enter minutes : 1
Enter seconds : 1
Hour : 1
minute : 1
second : 1
Total seconds : 3661
```

18/9

Experiment 2 :

1. WAP to declare a class "city" having data members as name & population. Accept this data for 5 cities & display name of the city having highest population.

→

```
#include <iostream>
using namespace std;
class city
{
 Private :
 String name;
 public :
 int pop ;
 void accept()
{
 cout<< " Name of city";
  Cin >> name;
 cout << " Population ";
  cin >> POP ;
 }
void display()
{
cout << " name of city : " << name ;
cout << " Population : " << POP ;
 }
};
 int main()
{
 city c[5];
```

```
int i, max=0;
for(i=0; i<5; i++)
{
c[i].accept();
}
for (i=1; i<5; i++)
{
if (c[i].pop)>(c[max].pop)
{
max = i;
}
cout << "\n city with highest population \n";
c[max].display();
}
}
```

Output:

```
name of city : Pune
population : 10000
name of city : Mumbai
population: 20000
name of city: Indore
Population: 30000
name of city: Nagpur
population : 40000
name of city : Kolkata
population : 50000

name of city with highest population:
Kolkata
```

b) WAP to declare a class 'Account' having data members as Account no & balance. Accept this data for 10 accounts & give interest of 10% where balance is equal or greater than 5000 & display them.

→

```
#include <iostream>
using namespace std;

class Account {
int acc no;
float balance;

public:
void accept() {
cout << " enter account Number :";
cin >> Acc No;
cout << " enter balance ";
cin >> balance;
}
void add interest()
{
if (balance >= 5000) {
balance = balance + (balance * 10 / 100);
}
}
void display()
{
cout << " Account No :" << acc no << " Balance"
<< balance << endl;
}
```

```
float get balance ()
{
return Balance;
}
};

int main () {
Account a[10];
for(int i=0; i<10; i++)
{
a[i] .accept ();
a[i] . add Interest ();
}
cout << "\n Account with Balance >=5000\n";
for(int i=0 ; i<10; i ++)
{
if (a[i] . get balance ()>=5000)
{
a[i] .display ();
}
}
Return 0;
}
```

Output:

```
Enter Account no: 1
Enter Balance:5000
Enter Acount n o: 2
Enter balance: 6500
Enter Account no:3
Enter Balance: 4000
Enter Account number :4
Enter balance : 8900
Enter Account no: 5
Enter balance : 3200
Enter Account no: 6
Enter balance : 7800
Enter Account no: 7
Enter balance : 1200
Enter account no: 8
Enter balance : 10000
Enter Account no:9
Enter balance : 2300
Enter account no:10
Enter balance: 4999

accounts with Balance >= 5000
account no: 1   Balance :5500
account no: 2   Balance : 7150
account no: 4   Balance : 8790
account no: 6   Balance : 3580
account no: 8   Balance : 10000
```

c) WAP to declare a class 'staff' having data members as name & post. Accept this data for 5 staff & display names of staff who are 'HOD'.

→

```
#include <iostream>
using namespace std;
class staff
{
String name, post;
```

```
public:
void accept()
{
cout<<"enter staff name:";
cin>>name;
cout<<"enter staff post:";
cin>>post;
}
void display()
{
cout<<"\n Staff Name:"<<name<<"\n Staff post:"
    <<post<<endl;
if(post=="HOD"||post=="Head")
{
cout<<name<<" is head of department \n";
}
else
{
cout<<name<<" is a staff \n";
}
}
};
int main()
{
Staff S[5]
Int i;
for(i=0;i<5;i++)
{
S[i].accept();
}
for(i=0;i<5;i++)
{
S[i].display();
}
return 0;
}
```

Q1

18/9

Experiment no 3 :

a) Wap to declare a class `Book` containing data members as book-title, author-name & price. Accept & display the info for one object using a pointer to that object.

→
```
#include <iostream>
using title;
string a_name;
int price;
public:
void info(){
cout << "enter the book title, author name & price of the book:";
cin >> title >> a_name >> Price;
}
void display(){
cout << "Book name:" << title << "\tauthor name:" << a_name << "\t price" << Price;
}}
};
int main(){
Book * n;
Book b1;
n -> info();
n -> display();
```

output:

Enter the Book title, author name & price of the book

Book 1
J.K rowling
675

Book name: Book 1 author name: J.K rowling
Price: 675.

2) WAP to declare a class "student" having data member roll_no & percentage using 'this' pointer

→
```
#include <iostream>
using name space std;
class student {
int roll;
float perc;
Public:
void info (int roll_no, float percentage){
this -> roll = roll_no;
this -> Perc = Percentage;
}
void display (){
cout<<"roll no:" << roll << "\t mark:" << Perc; }
}
};
int main(){
student s;
s.info(3, 90);
s.display();
}
```

output:

roll no: 3 marks: 90

3) Nested Class

→
```
#include <iostream>
using namespace std;
class marks {
public:
class percentage {
int marks, n;
float n, i;
public:
void info () {
cout << " enter the marks you got: ";
cin >> marks;
cout << " enter total marks: ";
cin >> n;
}
void display () {
i = (float) marks \n;
n = i * 100;
cout << " percentage: " << n;
} } };
int main () {
marks m1;
marks :: percentage n1;
n1 info ();
n1 . display ();
}
```

output:

Enter the marks you got : 67
Enter total marks : 100
Percentage : 95.71

18/9

Experiment no: 4

1. Swap 2 no.s from same using object as function argument.

→
```
#include <iostream>
using namespace std;
class number{
int value;
public:
number(int v=0){
value = v;
}
void swap(number &other){
int temp = value;
value = other.value;
other.value = temp;
}
void disp(){
cout<<" Before swap Value:"<<value<<endl;
}};
int main(){
number n1(10), n2(20);
cout<<" Before swap:"<<endl;
n1.disp();
n2.disp();
n1.swap(n2);
cout<<"\n After swap:"<<endl;
n1.disp();
n2.disp();
return 0;
}
```

Output:

Before swap:	After swap:
Value: 10	Value: 20
Value: 20	Value: 10

2. Swap 2 numbers from same class using friend function.

→
```
#include <iostream>
using namespace std;
class AB{
int A,B;
public:
void info(){
cout<<" enter 2 numbers:";
cin>>a>>b;
}
friend void swap(AB a1);
};
void swap(AB a1){
int temp;
temp = a1.a;
a1.a = a1.b
a1.b = temp;
cout<<"\n values after swapping :"<<a1.a<<a1.b;
}
int main(){
AB a1;
a1.info;
swap(a1);
}
```

output:
Enter 2 numbers: 5
4
values after swapping: 4
5

3. Find function swap 2 numbers diff class

→
```
#include <iostream>
using namespace std;
class CB;
class CA;
int numA;
public:
CA(int val): numA(val){}
void disp(){
cout<<"value in class A: "<<numA<<endl;
}
friend void swap(CA& CB &);
};
class CB{
private:
int numB;
CB(int val): numB(val){}
void disp(){
cout<<"value in class B: "<<numB<<endl;
}
friend void swap(CA &, CB &);
};
void swap(CA &a, CB &b){
int temp = a.numA;
a.numA = b.numb;
b.numB = temp;
}
int main(){
CA obj A(10);
CB obj B(20);
cout<<" Before swapping:"<<endl;
obj A.disp();
obj B.disp();
Swap(obj.A, obj.B);
cout<<"\n after swapping:"<<endl;
obj A.disp();
obj B.disp();
return 0;
}
```

output:
Before swapping:
value in class A: 10
value in class B: 20

After swapping:
value in class A: 20
value in class B: 10

4. Avg of two results.

→
```
#include <iostream>
using namespace std;
class result2;
class result{
int a;
```

```
public:
void accept(){
cout<<" enter marks out of 50:";
cin>>a;
}
friend void (a) (result x1, result x2);
};
class result2{
int b;
public:
void accept(){
cout<<"Enter marks out of 50;";
cin>>b;
}
friend void cal(result x1, result x2);
};
void cal(result x1, result x2){
float avg = (float)(x1.a + x2.b)/2;
cout<<"\navg:"<<avg;
}
int main(){
result x;
result2 y;
x.accept();
y.accept();
cal(x,y);
}
```

output:
Enter marks out of 50: 36
Enter marks out of 50: 38
avg: 37

5. Greatest among 2 numbers (Diff class)(Frnd Func)

→
```
#include<iostream>
using namespace std;
class B;
class A{
int a;
public:
void acc(){
cout<<" enter values:";
cin>>a;
}
frnd void gr(Aal, Bal);
};
class B{
int b;
Public:
void accept(){
cout<<"enter avalue:";
cin>>b;
}
frnd void gr(Aal, Bal);
};
void gr(Aal, Bal){
if(al.a > bl.b){
cout<<"First value is greater";
}
```

```
else{
cout << "second value is greater";
}
int main(){
A x;
B y;
x.acc();
y.acc();
gr(x,y);
}
```

output:
Enter value: 10
Enter value: 100
second value is greater

practice questions on friend func.

① Create two class co, class A & class B, each with a private integer. write a friend function sum() that access private data from both classes & return the sum.

→
```
#include <iostream>
using namespace std;
class classB;
class classA {
int a;
public:
classA(int val) : a(val) {}
friend int sum(classA, classB);
};
class classB {
int b;
public:
classB(int val) : b(val) {}
friend int sum(classA, classB);
};
int sum(classA objA, classB objB){
return objA.a + objB.b;
}
int main(){
classA a(10);
classB b(20);
cout<<"sum:" <<sum(a,b) <<endl;
return 0;
}
```

output:
Sum: 30

2. WAP with a class number that contains a private integer. Use a friend function swap nos (number &, number &) to swap the private values of two number objects.

→
```
#include <iostream>
using namespace std;
class number {
int value;
public;
Number (int val) : value(val) {}
Void display() { cout << value << endl; }
friend void swap numbers (Number &, Number &);
};
Void Swap Numbers (Number & n1, number & n2) {
int temp = n1.value;
n1.value = n2.value
n2.value = temp
}
int main () {
Number n1 (5), n2(15);
cout << "Before swap :"; n1.display(); n2.display();
Swap Numbers (n1, n2);
cout << "After swap :"; n1.display(); n2.display();
return 0;
}
```

output:
before swap: 5
15
after swap : 15
5

3. Define two classes Box & cube, each having a private volume. Write a friend func. findgreater (Box, cube) that determines which object has a larger volume.

→
```
# include <iostream>
using name space std;
class cube;
class Box {
int volume;
Public:
Box (int v) : volume (v) {}
friend void find Greatin (Box, cube);
};
class cube {
int volume;
public:
cube (int v) : volume (v) {}
Friend void find greatin (Box, cube);
};
Void find Greatin (Box b, cube c) {
cout << " Greates Volume :" << (b.volume > c.volume)
b.volume : c.volume) << endl;
}
```

```
int main (){
Box Box(118);
Cube Cube(90);
Find Greater(box, cube);
return 0;
}
```

output:
greater volume : 118

4. Create a class complex with real & imaginary parts as private members. Use a friend func to add two complex numbers & return the result as a new complex object.

→
```
#include <iostream>
using namespace std;
class complex{
int real, imag;
public:
complex (int r=0 , int i=0): real(r), imag(i){}
void display () {cout <<real << "+" << imag<<"i"
<<end l ;}
Friends complex add (complex , complex);
};
complex add (complex c1, complex c2){
return complex (c1.real + c2.real, c1.imag+
c2.imag);
}
```

```
int main (){
complex c1 (4,5) , c2 (2, 3) ;
complex result = add (c1, c2);
cout << "Sum of complex numbers:";
result display ();
return 0;
}
```

output:
Sum of complex numbers : 6 + 8i

5. Create a class student with private data members name & three subject marks. Write a friend function calculate average (student) that & display the avg marks.

→
```
#include <iostream>
using namespace std;
class Student {
int m1, m2, m3 ;
public:
Student (string n, int a , int b, int c), name(n),
m1 (a) , m2(b) , m3 (c) {}
Friends void calculate Average (student s){
float avg = (s.m1 +s.m2 + s.m3) /3
cout << " Average marks of " <<s.name <<" :"
<<avg <<end l ;
}
int main (){
Student s ("sai ", 90, 932, 91);
Calculate Average (s);
```

return 0;
}

output:
Avg marks of Sai : 91

6. Create 3 classes : Alpha, Beta, Gamma, each with a private data member. Write a single friend function that can access all three private data and find their sum.

→ #include <iostream>
using namespace std;
class Beta;
class Gamma;
class Alpha {
int a;
public:
Alpha(int val) : a(val) {}
friend void printSum(Alpha, Beta, Gamma);
};
class Beta {
int b;
Public:
Beta(int val) : b(val) {}
friend void printSum(Alpha, Beta, Gamma);
};
Class Gamma {
int c;
public:
Gamma(int val) : c(val) {}

friend void printSum(Alpha, Beta, Gamma);
};
void printSum(Alpha x, Beta y, Gamma z) {
cout << "Sum:" x.a + y.b + z.c << endl;
}
int main() {
Alpha a(10); Beta b(20); Gamma c(30);
printSum(a,b,c);
return 0;
}

Output:
Sum = 60

7. Create a class point with private members x & y. Write a friend func that calcs & returns the dist between 2 point objects.

→ #include <iostream>
#include <cmath>
using namespace std;
class point {
int x, y;
public:
point(int a, int b) : x(a), y(b) {}
friend double distance(Point, Point);
};
double distance(Point p1, Point p2) {
return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
}

```
int main() {
point P1(0,0), P2(3,4);
cout << "Dist:" << distance(P1,P2) << endl;
return 0;
}
```

output:
Distance: 5

8. Create two classes: Bank account & Audit. Bank account holds private balance info. Write a friend function in Audit that accesses & prints balance info for auditing.

```
→ #include <iostream>
using namespace std;
class BankAccount {
double balance;
public:
BankAccount(double b) : balance(b) {}
friend class Audit;
};
class Audit {
public:
void printBalance(BankAccount acc) {
cout << "Audited Balance: " << acc.balance
    << endl;
  }
};
```

```
int main() {
BankAccount acc(5000.75);
Audit audit;
audit.printBalance(acc);
return 0;
}
```

output:
Audited Balance: 5000.75

18/9

Assignment:

CH-2
Functions in C++

- Member function: The functions declared inside a class to operate on its members.

Syntax:

```
class MyClass {
int x;
void show();
};
```

- Passing Argument to functions:

1. Constant Argument: prevents modification of passed values.

Syntax:

```
void display(const int x);
```

2. Default Argument: Assigns default values if none given.

Syntax:

```
void greet(string name = "user");
```

- Passing Variables:

1. By value:

- Define: copies the actual value
- Syntax: `void update(int a);`

2. By Reference:

- Define: Passes address, allowing direct modifications
- Syntax: `void updata(int &a);`

- Nesting of member function:

One member function calls another within the same class.

Syntax:

```
Class test {
void helper();
public:
void main function() {
helper();
}
};
```

- Returning values from function

1. Return Statement: Ends function & sends value back

Syntax:
```
int add (int a, int b){
return a+b;
}
```

2. Return by Reference: Returns reference to variable.

Syntax:
```
int & get value (){
static int x=10;
return x;
}
```

3. Returning object: Returns an entire object

Syntax:
```
My class create obj (){
my class obj;
Return obj;
}
```

- Friend function:

External function with access to private members of a class.

Syntax:
```
class My Class{
int x;
friend void show (My Class);
};
Void show (My Class obj){
cout << obj.x;
}
```

- Pointer to object:

pointer that stores address of an object

Syntax:
```
My class * Ptr = new My class;
Ptr → display();
```

- this Pointer:

Implicit pointer that refers to the current object

Syntax:

```
class MyClass {
int x;
public:
void setx (int x) {
this -> x = x;
}
};
```

- Nested class:

class defined inside another class

Syntax:

```
class outer {
Public:
class Inner{
Public:
void show() {
cout<< "Inner class";
}
};
};
```

very short note

Experiment no - 5:

a) Write a program to find the sum of no. between 1 to n using a constructor where the value of n will be passed to the constructor.

→ Default constructor:

```
#include <iostream>
using namespace std;
class add{
    int total = 0, n, i;
public:
add() {
 n=5;
}
void calculate (){
 total =0;
 for(i=1 ; i<=n ; i++)
{
 total = total + i;
}
cout <<"Sum of first 5 numbers is:"<<total <<endl;
}};
int main (){
add s1;
s1.calculate();
return 0;
}
```

O/P

Sum of first 5 numbers is 15.

→ copy constructor :

```
#include<iostream>
using namespace std;
class add{
  int total=0,n,i;
public:
add(int num){
 n=num;}
add(add &obj){
n=obj.n;}
void calculate(){
 total=0;
for(i=1,i<=n,i++){
total=total+i;
}
cout<<"sum of 1 to n no. is:"<<total<<endl;
}};
int main(){
int n;
cout<<"enter value of n:";
 cin>>n;
add s1(n);
add s2(s1);
s2.calculate();
return 0;
```

O/P :

enter value of n : 5

sum of 1 to n numbers is : 15

→ Parametrized constructor

```
#include<iostream>
using namespace std;
class add{
 int total=0,n,i;
public:
add(int num){
   n=num;
}
void calculate(){
total=0;
for(i=1,i<=n,i++){
 total=total+i;
}
cout<<"sum of 1 to n numbers is:"<<total<<endl;
}};
int main(){
int n;
cout<<"enter the value of n:";
cin>>n;
add s1(n);
add s1(n);
s1.calculate();
return 0;
}
```

O/P:

Enter the value of n : 5

Sum of 1 to n numbers is : 15

b). W.AP to declare a class student having data members as name & percentage. Write a const. to initialize these data members, accept & display data for 1 student.

→ Default const:

```
#include <iostream>
using name space std;
class student{
 int per ;
 string nam;
public :
  student(){
       per =90;
       name ="arnav";
}
void display (){
cout<< "Name : "<<name<<endl;
cout<< "percentage: "<<per<<endl;
};
int main (){
int per;
string name;
student s1;
s1.display ();
return 0;
}
```

Output

Name = sai

Percentage = 85

→ Copy Const :

```
#include <iostream>
using name space std;
class student{
int per ;
string name;
public :
student (int p, string n){
    per =P ;
    name =n;
}
Student (student &s){
      per =s.per;
      name =s.name;
}
void display (){
 cout<< "name : "<< name<< endl;
 cout<< "Percentage :" <<per<<endl;
};
int main ()
int per;
string name;
cout<<"enter name";
cin >> name;
cout<< "enter percentage : ";
 cin >> Per ;
Student s1 (per, name);
Student s2(s1);
 s2.display;
return 0;
}
```

O/P:
Enter name : sai
Percentage : 85

→ Para metrized const:

```
#include <iostream>
using namespace std;
class student {
  int per;
  string name;
public:
student (int p, string n){
  per = p;
name = n;
}
void display (){
cout<<"name:"<<name<<endl;
cout<<"Percentage:"<<per<<endl;
}};
int main (){
int per;
string name;
cout<<"enter name:";
cin>>name;
cout<<"enter percentage:";
cin>>per;
student S1(per, name);
S1.display();
return 0;}
```

O/P: enter name: sai
enter percentage: 85

c) WAP to demonstrate const overloading.

→
```
#include <iostream>
using namespace std;
class rectangle {
    int l, w;
  public:
    Rectangle () {
      l=1;
      w=2; }
Rectangle (int a){
    l=a;
    w=a; }
Rectangle (int a, int b){
      l=a;
      w=b; }
void calculate (){
    int a;
    a = lxw
    cout<<"area ="<<a;
};
int main (){
Rectangle r1;
Rectangle r2 (5);
Rectangle r3(4,5);
r1. calculate ();
r2. calculate ();
r3. calculate (); }
```

O/P: area = 2
area = 25
area = 20

12/11

Experiment no-6:

Write a C++ program to implement inheritence

- Single inheritence:

```
#include <iostream
using namespace std;
class person{
    protected:
    string name;
    int age;
};
Class student : Protected person
int roll_no;
Public:
void accept () {
cout<<"enter name:";
cin>>name;
cout<<"enter age:";
cin>>age;
cout<<"enter roll_no:";
cin>>roll_no;
}
void display (){
cout<<"name:"<<name<<endl;
cout<<" Age :"<<age<<endl;
cout<<" Roll no:"<<roll_no<<endl;
};
int main(){
Student S;
S.accept();
S.display();
return 0;
}
```

O/P:

```
enter name: Sai
enter age :17
enter roll no: 26
name :Sai
age :17
rollno : 26
```

- Multiple inheritence :

```
#include <iostream>
using namespace std;
Class academic {
protected:
string name;
int marks;
};
class sports {
protected;
int score;
};
class student : protected academic , protected sports {
int total;
public :;
void accept () {
cout<<"enter name:";
cin>>name;
cout<<"enter marks";
cin>>marks;
```

```
cout << "enter score";
cin >> score;
total = marks + score;
}
void display () {
cout << "name : " << name << endl;
cout << "marks: " << marks << endl;
cout << "score :" << score << endl;
cout << "total" << total << endl;
};
int main () {
student s;
s.accept();
s.display();
return 0;
}
```

O/P:

```
Enter name : Sai
Enter marks : 60
Enter score : 40
name : Sai
marks : 60
score : 40
total : 100
```

- Multilevel inheritence:

```
#include (iostream)
using namespace std;
class vehicle {
Public:
String model;
String brand;
};
class electric car : public car {
  Public:
int battery capacity;
void accept() {
cout << "enter model:";
cin >> model;
cout << "enter brand";
cin >> brand;
cout " Enter type (1 for sedan, 2 for suv):";
cin >> type;
cout << enter battery capacity (in kwh):";
cin >> battery - capacity;
}
void display() {
cout << "Model : " << model << endl;
cout << " Brand : " << Brand << endl;
cout << " type : " << (type == 1 "sedan" : "suv") << endl;
cout << " Battery capacity : " << battery - capacity << "kwh" << endl;
};
int main () {
electric car c;
c.accept();
c.display ();
return 0;
}
```

O/P:

Enter model : M++	model : M++
Enter brand : XYZ	brand : XYZ
enter type : 2	type : suv
Enter battery : 44 KWh	capacity : 44 KWh